AI Research Copilot

PROJECT DOCUMENTATION

This guide describes the application as implemented in its frontend, FastAPI backend, configuration, dependency manifests, and startup scripts.

Table of contents

1. Overview 3

2. Technology stack 3

3. Models 4

4. System architecture 4

4.1 Paper indexing and category assignment . . . 5

4.2 Local storage . . . 6

5. Page workflows 6

5.1 Research Chat . . . 6

5.2 Paper Analysis . . . 7

5.3 Research Gaps . . . 8

5.4 Paper Repository . . . 9

5.5 Paper Comparison . . . 10

6. API reference 11

7. Project folder structure 12

8. How to set up and run 12

8.1 First-time setup on Windows . . . 12

8.2 Daily startup . . . 12

8.3 Regenerate this documentation . . . 13

8.4 Service links . . . 13

9. Seeded data and configuration 13

10. Evidence boundaries 13

1. Overview

AI Research Copilot is a local application for indexing research papers in PDF format, searching their contents, and generating paper-grounded answers and reports. The React and Vite frontend calls a FastAPI backend. The backend uses a local Sentence Transformers embedding model, a FAISS vector index, SQLite metadata, and an Ollama language model.

The checked-in seed corpus contains seven sample PDFs. Running the seed command builds an index with 173 chunks and stores stable sample-paper categories. New uploads are extracted, classified, and indexed through the same application.

2. Technology stack

The versions below come from the project dependency pins and the project setup guide. Ollama's model tag is a model name, not a package version.

Technology	Version	Where used	Why
Python	3.14.0	Backend and seed scripts	Runs the API, paper processing, embeddings, and seed workflow
Node.js	24.11.1	Root scripts and frontend tooling	Runs startup, setup, documentation, and Vite scripts
npm	11.6.2	Root and frontend package managers	Installs the pinned JavaScript dependencies
React and React DOM	19.3.0	`frontend/src`	Build the interactive application pages
lucide-react	1.45.0	Frontend page components	Supplies the interface icons
Vite	8.3.0	`frontend`	Serves the frontend and proxies `/api` requests to FastAPI
Mermaid	12.2.0	Paper Analysis frontend	Validates and renders generated architecture diagrams
FastAPI	0.161.1	`backend/app.py`	Defines the HTTP API and request validation
Pydantic	2.14.1	`backend/app.py`, `backend/schemas.py`	Validates API request and response data
Uvicorn	0.54.0	Root development launcher	Serves the FastAPI application on `127.0.0.1:8000`
python-multipart	0.0.32	FastAPI PDF upload	Parses uploaded PDF form data
FAISS CPU	1.15.1	`backend/research_copilot.py`	Stores and searches normalized paper-chunk vectors
Sentence Transformers	6.2.0	`backend/research_copilot.py`	Loads the local embedding model and creates text embeddings
NumPy	2.5.4	Backend retrieval and category logic	Holds and normalizes embedding vectors
PyMuPDF	2.0	Backend PDF processing	Extracts PDF text and detects or renders original figures
SQLAlchemy	2.1.6	`backend/database.py`, `backend/models.py`	Reads and writes relational metadata
python-dotenv	1.2.4	`backend/config.py`	Loads project environment settings from `.env` files
SQLite	Python standard library	`data/research_copilot.db` by default	Persists indexed-paper and research-category metadata
Ollama	0.40.10 in the setup guide	Local LLM service at port 11434	Serves the configured local language model
qwen2.5:3b	Model tag	Backend LLM requests	Generates answers, analysis, category proposals, and reports

Technology	Version	Where used	Why
all-MiniLM-L6-v2	384 dimensions	Local `model/embedding_model`	Embeds paper chunks and category classification text
ReportLab	5.1.0	Backend PDF export and this documentation generator	Creates analysis PDFs and the project documentation PDF
concurrently	9.2.5	Root `npm run dev`	Runs the local services together in one terminal
wait-on	8.0.5	Root startup scripts	Waits for the frontend and backend endpoints to become ready

3. Models

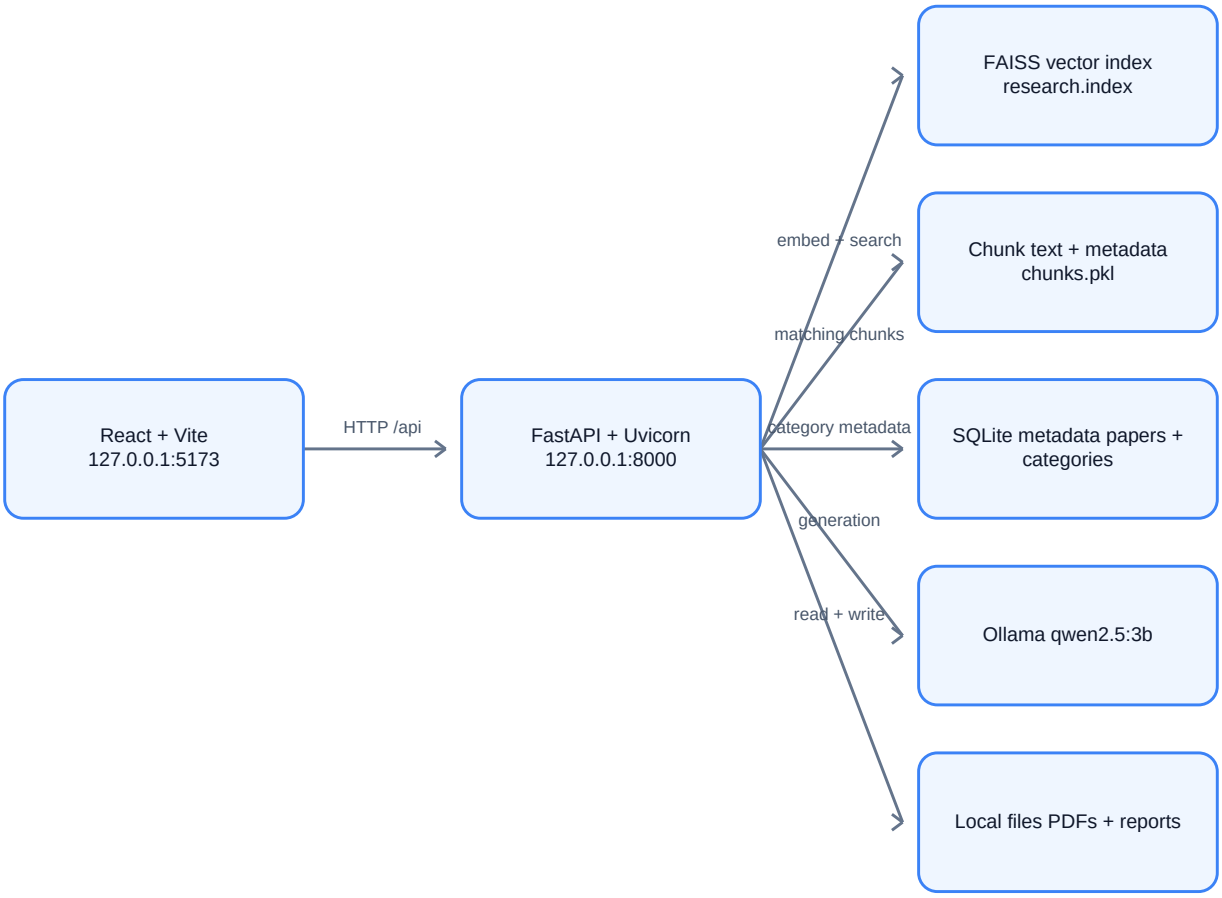
Model	Type	Purpose
sentence-transformers/all-MiniLM-L6-v2	Text embedding model, 384-dimensional vectors	Encodes paper chunks and classification text so FAISS can retrieve related passages and category centroids can be compared
qwen2.5:3b	Local Ollama language model	Produces library-grounded chat answers, paper analysis, research gaps, comparisons, literature reviews, citations, and category proposals

The embedding model is loaded from `model/embedding_model`. Ollama is configured with the `qwen2.5:3b` tag and its local service URL comes from `OLLAMA_URL`.

4. System architecture

The browser uses Vite's `/api` proxy to reach FastAPI. FastAPI coordinates retrieval and generation. FAISS stores vectors; `chunks.pkl` stores the chunk text and per-chunk metadata aligned to the vectors. SQLite stores paper categories, classification text, page counts, centroid vectors, and category counts. The paper PDFs and generated files live in local project folders.

Figure 1. Main application components and local data flow.



4.1 Paper indexing and category assignment

1. The upload endpoint accepts a PDF and saves it under `research_papers`.
2. PyMuPDF extracts text page by page. The backend chunks the text, creates 384-dimensional embeddings, normalizes them, and appends the vectors to FAISS.
3. Category text is built from the paper's title, abstract, keywords, and introduction where those sections can be identified in the opening pages.
4. The embedding of that classification text is compared with the normalized centroid of each existing category. The default threshold is `0.55`, configurable with `CATEGORY_SIMILARITY_THRESHOLD`.
5. If no category reaches the threshold, Ollama proposes a short category name. Known aliases and capitalization are normalized before the category is saved.
6. The paper's category and classification text are stored with the FAISS chunk metadata and in SQLite. Category centroids and paper counts are recalculated from indexed paper embeddings.

The seed workflow uses the checked-in category mapping instead of asking Ollama to classify the seven sample PDFs. This keeps the sample categories and 173-vector count reproducible.

4.2 Local storage

Path	Contents
`sample_papers/`	Seven PDFs used by the seed workflow
`research_papers/`	PDFs currently available to the index and paper viewer
`extracted_text/`	Runtime directory created by configuration; indexed chunk text is stored in `vector_db/chunks.pkl`
`vector_db/research.index`	FAISS inner-product vector index used with normalized vectors
`vector_db/chunks.pkl`	Chunk text and metadata corresponding to FAISS entries
`data/research_copilot.db`	SQLite database; includes indexed-paper and category metadata
`outputs/`	Generated text and JSON reports
`model/embedding_model/`	Checked-in local embedding model files

The runtime data directories are created by backend configuration. Generated indexes and the local database are not required in Git; `npm run seed` builds them from the checked-in PDFs and model.

5. Page workflows

5.1 Research Chat

Research Chat shows the live count of indexed papers, supports a target-paper filter and category filter, and offers Quick, Balanced, and Deep search depth (3, 5, and 10 retrieved chunks). Its normal target corpus is all indexed papers. The backend returns a fixed not-found response when retrieved context is absent or below the configured RAG similarity threshold. Supported answers are generated from retrieved passages and include paper and page citations.

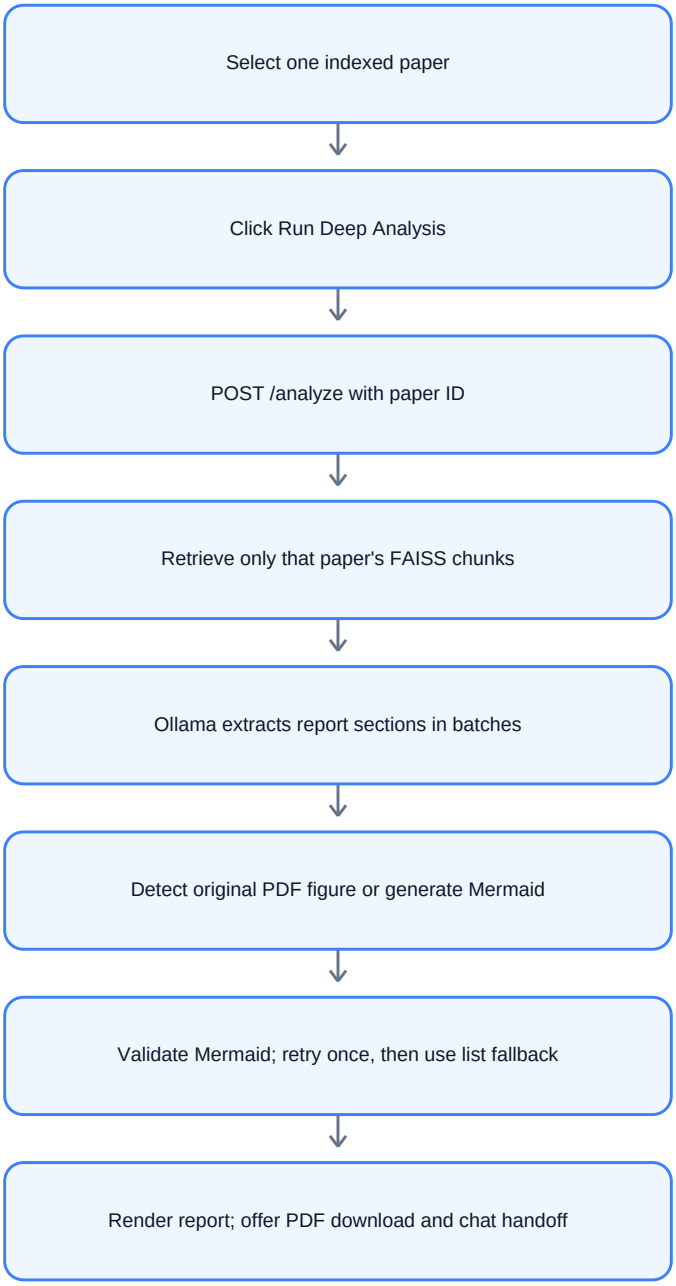
Figure 2. Research Chat request and answer workflow.

5.2 Paper Analysis

The user selects one indexed paper and starts deep analysis. The backend filters the chunk metadata to that paper, sends its chunks to Ollama in small batches, and produces the report sections listed below. Missing sections are presented as "Not mentioned in the paper." The architecture view uses an extracted original PDF figure when the backend detects a likely architecture figure; otherwise the backend asks Ollama for a paper-based Mermaid diagram. The frontend validates generated Mermaid and asks the backend for one retry before falling back to a numbered list. The page can download a PDF report and open Research Chat with that paper selected.

Report sections: Problem Statement, Proposed Method, System Architecture, Mathematical Models, Dataset, Metrics, Results, Strengths, Limitations, and Future Work.

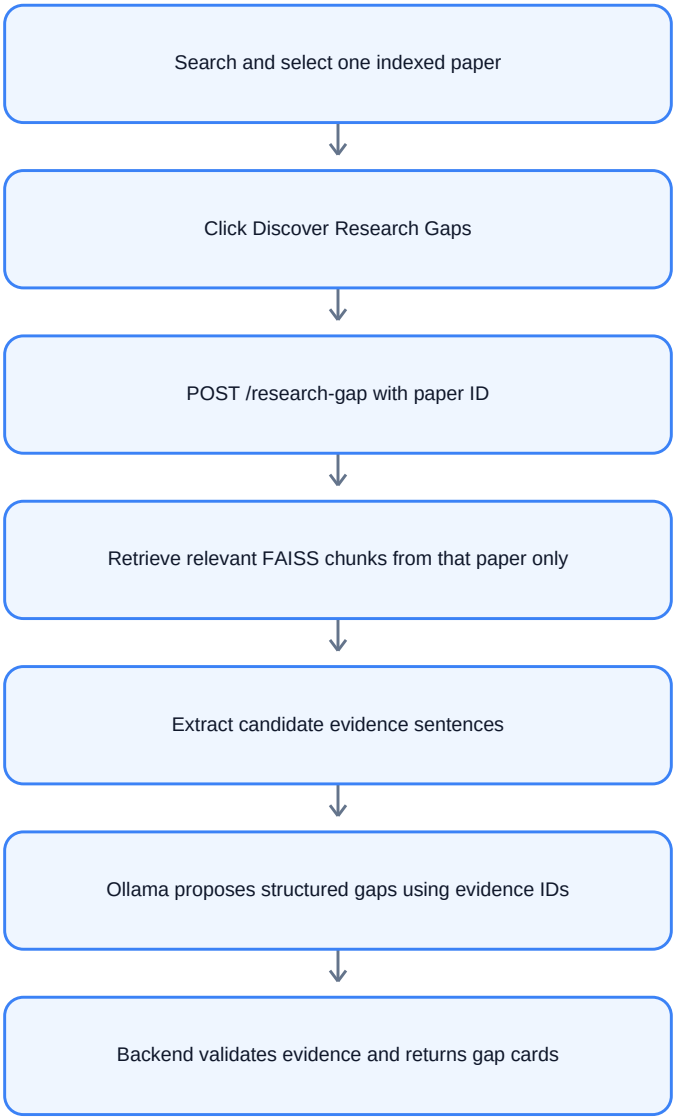
Figure 3. Single-paper analysis workflow.



5.3 Research Gaps

The user selects one paper from searchable paper cards. The button remains disabled until selection. The backend searches for relevant passages across several limitation and challenge queries, then filters matches to the selected paper. It extracts evidence sentences, asks Ollama for structured gaps, and accepts only results that refer to supplied evidence IDs. Each displayed gap includes a title, explanation, severity, research idea, and supporting evidence.

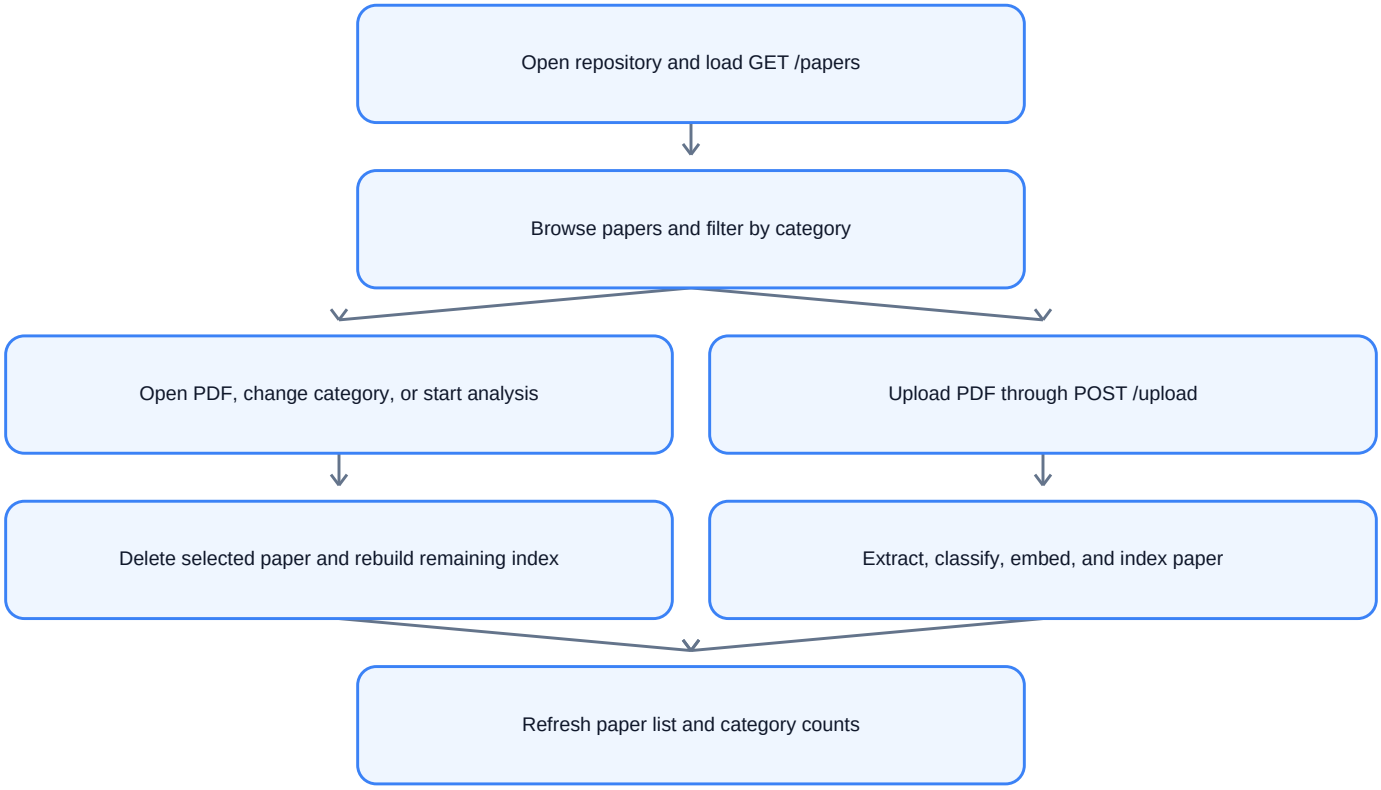
Figure 4. Selected-paper research gap workflow.



5.4 Paper Repository

The repository lists indexed PDFs, pages, categories, and chunk counts. It groups the list by category, allows category filtering and manual category changes, opens the PDF, starts analysis for a selected paper, and deletes a paper from the index and local file storage. Uploading a PDF starts extraction, classification, and indexing in the backend.

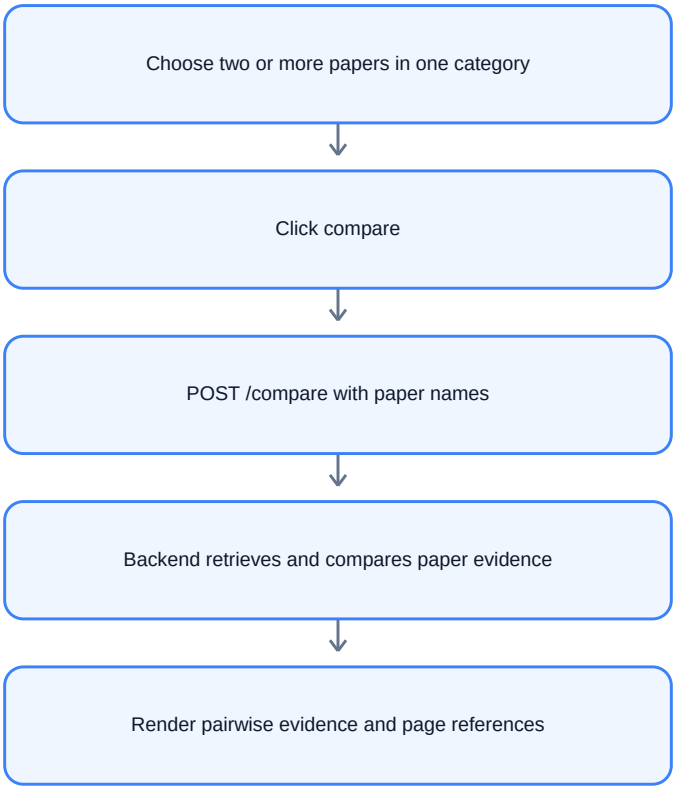
Figure 5. Paper Repository and upload workflow.



5.5 Paper Comparison

The comparison page groups papers by category, lets the user select papers from the same category, and sends the selection to the backend. The backend compares supported evidence fields and returns pairwise results. The frontend shows the evidence by paper, including page references where returned.

Figure 6. Paper Comparison workflow.



6. API reference

The Vite frontend sends requests through `/api`; Vite removes that prefix and forwards them to FastAPI. The endpoint paths in this table are the FastAPI paths.

Method	Endpoint	Purpose
GET	`/`	Return API status, engine health, and the OpenAPI docs path
GET	`/health`	Report model, FAISS, paper, and Ollama status
GET	`/papers`	List indexed papers and category counts
PATCH	`/papers/{paper_name}/category`	Update or create a paper category
PATCH	`/papers/{paper_name}/sector`	Compatibility alias for category updates; hidden from the OpenAPI schema
POST	`/upload`	Extract and index an uploaded PDF
DELETE	`/papers/{paper_name}`	Remove a paper from the index and optionally delete its PDF
GET	`/paper-file/{paper_name}`	Serve an indexed PDF for viewing
POST	`/search`	Search indexed chunks with optional filters
POST	`/ask`	Answer a question from retrieved paper passages
POST	`/analyze`	Generate a report for one indexed paper
POST	`/analysis/architecture`	Retry generated architecture Mermaid
POST	`/analysis/pdf`	Build and download an analysis PDF
POST	`/compare`	Compare two or more paper names
POST	`/research-gap`	Detect evidence-grounded gaps in one paper
POST	`/literature-review`	Generate a literature review for selected or all papers
POST	`/citation`	Generate a citation in the requested style
GET	`/outputs`	List generated text reports
GET	`/outputs/{filename}`	Download a generated text report

The backend also contains authentication and user route modules, but `backend/app.py` does not register those routers. They are therefore not listed as active API endpoints.

7. Project folder structure

```
AI-Research-Copilot/
|-- backend/
|   |-- app.py
|   |-- config.py
|   |-- database.py
|   |-- models.py
|   |-- research_copilot.py
|   |-- requirements.txt
|   |-- seed.py
|   |-- routes/
|   `-- scripts/
|-- docs/
|   |-- PROJECT_CONTENT.md
|   |-- generate_pdf.py
|   `-- AI_Research_Copilot_Documentation.pdf
|-- frontend/
|   |-- package.json
|   `-- src/
|       |-- App.jsx
|       |-- services/
|       `-- components/
|-- model/embedding_model/
|-- sample_papers/
|-- scripts/
|-- data/
|-- extracted_text/
|-- outputs/
|-- research_papers/
|-- vector_db/
|-- .env.example
|-- package.json
|-- package-lock.json
`-- seed_manifest.json
```

The first four runtime data folders are created as needed. The embedding model and sample PDFs are project inputs; the database, FAISS index, chunks file, and generated reports are local runtime outputs.

8. How to set up and run

8.1 First-time setup on Windows

Install the versions listed in the project README, then run the following commands from the project root in PowerShell:

```
npm run setup
ollama pull qwen2.5:3b
npm run seed
```

`npm run setup` installs the locked root and frontend packages, creates `.venv` with Python 3.14 if needed, installs the pinned backend dependencies, and copies `.env.example` to `.env` if no local `.env` exists. `npm run seed` checks the PDFs and embedding model against their SHA-256 values, indexes the seven sample papers, and checks the expected 173 vectors.

8.2 Daily startup

Run this from the project root in one terminal:

```
npm run dev
```

The root launcher starts or reuses Ollama, checks for `qwen2.5:3b`, starts FastAPI on `http://127.0.0.1:8000`, starts Vite on `http://127.0.0.1:5173`, waits for both web services, and opens the frontend in the default browser. Press Ctrl+C in that terminal to stop the project processes. `npm run stop` is also available.

8.3 Regenerate this documentation

After `npm run setup`, run the following from the project root:

```
npm run docs
```

The command reads `docs/PROJECT_CONTENT.md` and writes `docs/AI_Research_Copilot_Documentation.pdf`. The PDF uses the pinned ReportLab dependency from the backend virtual environment.

8.4 Service links

Service	URL
Frontend	`http://127.0.0.1:5173`
Backend health	`http://127.0.0.1:8000/health`
Interactive API documentation	`http://127.0.0.1:8000/docs`
Ollama service	`http://127.0.0.1:11434`

9. Seeded data and configuration

The seed manifest defines the seven sample PDF names, SHA-256 values, category mapping, per-paper chunk counts, 490-word chunk size, 100-word overlap, and expected total of 173 vectors. The seed checks the included embedding model checksum as well. It refuses to seed over unrelated indexed papers or a different PDF already present at a sample-paper path.

Useful local settings from `.env.example`:

Setting	Default	Use
`DATABASE_URL`	`sqlite:///data/research_copilot.db`	SQLAlchemy database connection
`OLLAMA_URL`	`http://127.0.0.1:11434`	Local Ollama server
`OLLAMA_MODEL`	`qwen2.5:3b`	LLM tag used by backend requests
`CATEGORY_SIMILARITY_THRESHOLD`	`0.55`	Minimum cosine similarity for existing-category assignment
`RAG_MIN_COSINE_SIMILARITY`	`0.30`	Minimum retrieved similarity accepted for chat context
`FRONTEND_URL`	`http://127.0.0.1:5173`	Frontend base URL configuration

10. Evidence boundaries

Research Chat constructs its prompt from retrieved indexed-paper passages and asks the model to cite those passages. If no passage reaches the RAG similarity threshold, the backend returns "Not found in your indexed papers." Paper Analysis filters chunks by the selected paper before calling the model. Research Gaps similarly filters its retrieved passages to the selected paper and validates that generated gaps cite supplied evidence.

The project uses a generative model, so response wording can vary. The indexed source papers, category mapping used by the seed, FAISS vector count, and citation source pages are determined by the local indexed data and seed manifest.